Inventory or High-Value Asset Risk Oracle: A Hardware-Anchored Assessment System

Authors: Sanjeev Jha and Jasreen Bagga

Project Overview

This project provides a functional blueprint for solving the "Oracle Problem" in decentralised finance. In traditional asset-based lending, physical collateral is monitored via intermittent human audits, which creates a vulnerability window for fraud.

This system replaces human oversight with a deterministic, hardware-anchored oracle. By continuously monitoring the physical mass of high-value assets using an IoT edge computing device, physical changes are instantly converted into digital triggers. If collateral is removed, the system captures visual proof and deterministically updates a smart contract on the Ethereum blockchain, completely automating the risk escalation process.

Core Features

- **Continuous IoT Monitoring:** A physical load cell actively tracks collateral mass in real-time, scanning for tampering or depletion.
- **Algorithmic Noise Filtering:** Localized data sanitization filters out environmental warehouse noise and vibrations before making on-chain reports.
- **Automated Smart Contract Logic:** An Ethereum smart contract natively evaluates the Loan-to-Value (LTV) ratio and automatically escalates the risk state (ACTIVE, FLAGGED_L1, FLAGGED_L2, FLAGGED_L3).
- **Sensor-Triggered Visual Proof:** The exact moment a weight drop is detected, the system captures a high-resolution photo and binds it to the blockchain record using a cryptographic SHA-256 hash.
- **Dual-State UI Dashboard:** A Flask-powered (python) web interface that provides zero-latency local telemetry alongside the official on-chain consensus.

Technical Stack & Hardware

- **Edge Computing:** Raspberry Pi 4
- **Sensors:** 5 kg Load Cell Sensor, HX711 Analog-to-Digital Converter, Raspberry Pi Camera Module
- **Backend / Middleware:** Python 3, Flask, Web3.py
- **Blockchain:** Solidity, Ethereum Sepolia Testnet
- **Frontend:** HTML, CSS, JavaScript (Asynchronous Polling)

File Directory Breakdown

1. Smart Contract

- **`Contract.txt`**: The core Solidity smart contract (`CollateralMonitor`). It stores the pledged baseline, processes updates from the oracle, deterministically calculates the health ratio, and emits events for the web dashboard. This has been implemented on the Ethereum interface.

2. Edge Middleware (Python)

- **`App.py`**: The main execution loop. It runs a background thread to monitor the physical weight sensor and simultaneously serves the Flask web dashboard.
- **`chain.py`**: The Web3 communication bridge. Handles the cryptographic signing of payloads and executes gas-consuming state transitions on the Sepolia Testnet.
- **`weight.py`**: The physical telemetry layer. Interfaces directly with the HX711 chip to acquire raw electrical signals, filter out outliers, and compute the mass in grams.
- **`camera.py`**: Triggers the Raspberry Pi camera to capture real-time visual evidence (proof of reserve) the moment a weight drop is detected.

3. Setup & Utilities

- **`test_weight.py`**: A standalone calibration script used to determine the `ZERO_OFFSET` (tare) and `CALIBRATION_FACTOR` for the physical load cell.
- **`ACC.py`**: A local cryptographic utility script that derives the public Ethereum wallet address from a raw private key.

4. Presentation Layer

- **`index.html`**: The frontend UI dashboard. It asynchronously polls the edge API to display real-time sensor weight, the official smart contract state, dynamic LTV health bars, and an immutable transaction audit trail.

Setup Instructions

1. **Hardware Assembly:** Connect the HX711 to the Raspberry Pi GPIO pins (DT=5, SCK=6) and wire the load cell to the HX711. Connect the Raspberry Pi Camera module.
2. **Calibration:** This file is specifically for calibrating the weight for the weight sensor with a known precision weight to calculate your specific hardware offsets. You can run it individually `python3 test_weight.py` or update `weight.py` with new values. This file is imported in the main App.py so that calibration runs at the start of the server.
3. **Environment Variables:** Securely set your `PRIVATE_KEY` in your environment to fund gas fees on the Sepolia test network.
4. **Requirements.txt** contains Python packages used throughout the project.
5. **Deploy Contract:** Deploy `Contract.txt` to Sepolia and update the `CONTRACT_ADDRESS` inside `chain.py`.
6. **Run the Oracle:**
    a. Install python libraries:
        i. Make sure you have python 3 installed in system
        ii. Make sure you are on the home path where requirements.txt file lies
        iii. Install libraries using command: `pip3 install -r requirements.txt`
    b. Execute `python3 App.py` to start the background monitoring loop and the Flask dashboard. Access the dashboard via `localhost:5000`.